République Algérienne Démocratique et Populaire

Université des Sciences et de la Technologie Houari Boumediene
Faculté d'Informatique

Master 2 Ingénierie du Logiciel – Année 2026/2027

Rapport de TP2

Entrepôts de Données

Les vues matérialisées

SGBD Oracle 19c – schéma 3ASSIR du TP1

Réalisé par : Abdelhadi DEFFOUS
Groupe 3

Module : Systèmes décisionnels et entrepôts de données (SDED)

Introduction

Ce TP porte sur les vues matérialisées. Contrairement à une vue classique, qui est recalculée à chaque utilisation (TP1), une vue matérialisée stocke le résultat de sa requête. Elle doit donc être rafraîchie quand les tables sources changent, soit par recalcul complet (COMPLETE), soit de façon incrémentale (FAST) grâce à un journal des modifications. Toutes les manipulations sont faites avec le compte Master sur le schéma du TP1, avec `SET TIMING ON` pour mesurer les temps.

1 VM1 : commandes de juin 2024 (COMPLETE)

VM1 est créée avec les options IMMEDIATE, COMPLETE et ON DEMAND. Le mois est filtré par un intervalle de dates (`DateCom >= '01/06/2024' AND DateCom < '01/07/2024'`) plutôt que par une fonction sur la colonne.

```
SQL> SET TIMING ON
SQL> SET LINESIZE 200
SQL> SET PAGESIZE 100
SQL> CREATE MATERIALIZED VIEW VM1
  2  BUILD IMMEDIATE
  3  REFRESH COMPLETE ON DEMAND
  4  ENABLE QUERY REWRITE
  5  AS
  6  SELECT CodeCom, DateCom, Prix, CodeClient, CodeDep, CodeArr, CodeV
  7  FROM   Commande
  8  WHERE  DateCom >= DATE '2024-06-01'
  9  AND    DateCom <  DATE '2024-07-01';

Vue matÚrialisÚe crÚÚe.

EcoulÚ : 00 :00 :02.82
SQL> SELECT COUNT(*) FROM VM1;

  COUNT(*)
----------
     48027

EcoulÚ : 00 :00 :00.01
```

Fig. 1 : Création de VM1

```
SQL> SELECT mview_name, build_mode, refresh_method, refresh_mode, rewrite_enabled
  2  FROM   user_mviews;

MVIEW_NAME   BUILD_MODE REFRESH_ME REFRESH_ REWRITE_
------------ ---------- ---------- -------- --------
VM1          IMMEDIATE  COMPLETE   DEMAND   Y

EcoulÚ : 00 :00 :00.00
SQL>
```

Fig. 2 : Options de VM1 dans USER_MVIEWS

La vue est créée en 2,82 s et contient 48 027 commandes. Ce temps correspond à la lecture de la table Commande (2,3 millions de lignes) pour remplir la vue dès sa création.

2 VM2 : même vue en FAST

Une première tentative de création en FAST échoue :

```
SQL> CREATE MATERIALIZED VIEW VM2
  2  BUILD IMMEDIATE
  3  REFRESH FAST ON DEMAND
  4  ENABLE QUERY REWRITE
  5  AS
  6  SELECT CodeCom, DateCom, Prix, CodeClient, CodeDep, CodeArr, CodeV
  7  FROM   Commande
  8  WHERE  DateCom >= DATE '2024-06-01'
  9  AND    DateCom <  DATE '2024-07-01';
AND    DateCom <  DATE '2024-07-01'
               *
ERREUR Ó la ligne 9 :
ORA-23413: la table "MASTER"."COMMANDE" n'a pas de journal des vues matÚrialisÚes


EcoulÚ : 00 :00 :00.02
SQL>
```

Fig. 3 : Création de VM2 sans journal

L'erreur ORA-23413 indique que la table Commande n'a pas de journal. Sans journal, Oracle ne peut pas savoir quelles lignes ont changé, donc un rafraîchissement incrémental est impossible. On crée le journal avec `CREATE MATERIALIZED VIEW LOG ON Commande;`, ce qui crée la table `MLOG$_COMMANDE`. Par défaut, elle ne stocke que la clé primaire des lignes modifiées (CodeCom), avec des colonnes techniques : type d'opération (DMLTYPE$$), ancienne ou nouvelle valeur (OLD_NEW$$), date de prise en compte (SNAPTIME$$), etc.

```
SQL> CREATE MATERIALIZED VIEW LOG ON Commande;

Journal de vue matÚrialisÚe crÚÚ.

EcoulÚ : 00 :00 :00.12
SQL> DESC MLOG$_COMMANDE
 Nom                                                
```

mais elle reste déclarée en COMPLETE.

3 Effet des mises à jour sur VM1 et VM2

On choisit une commande de juin 2024 à modifier (99) et une à supprimer (2340729). La commande supprimée est choisie au-delà de 1 822 210 pour qu'elle n'ait pas de course, sinon la clé étrangère de Course bloquerait la suppression.

```
SQL> SELECT MIN(CodeCom) AS a_modifier
  2  FROM   Commande
  3  WHERE  DateCom >= DATE '2024-06-01' AND DateCom < DATE '2024-07-01';

A_MODIFIER
----------
        99

EcoulÚ : 00 :00 :00.01
SQL> SELECT MAX(CodeCom) AS a_supprimer
  2  FROM   Commande
  3  WHERE  DateCom >= DATE '2024-06-01' AND DateCom < DATE '2024-07-01'
  4  AND    CodeCom > 1822210;

A_SUPPRIMER
-----------
    2340729

EcoulÚ : 00 :00 :00.00
SQL>
```

Fig. 6 : Choix des commandes à modifier et à supprimer

On ajoute ensuite une commande (2340931), on augmente le prix de la commande 99 de 1000 et on supprime la commande 2340729.

```
SQL> SELECT CodeCom, Prix FROM VM1 WHERE CodeCom IN (99, 2340729, 2340931);

   CODECOM       PRIX
---------- ----------
        99      17099
   2340729        681

EcoulÚ : 00 :00 :00.00
SQL> SELECT CodeCom, Prix FROM VM2 WHERE CodeCom IN (99, 2340729, 2340931);

   CODECOM       PRIX
---------- ----------
        99      17099
   2340729        681

EcoulÚ : 00 :00 :00.00
SQL> INSERT INTO Commande VALUES (2340931, DATE '2024-06-15', 5000, 1, 1, 1, 1);

1 ligne crÚÚe.

EcoulÚ : 00 :00 :00.02
SQL> UPDATE Commande SET Prix = Prix + 1000 WHERE CodeCom = 99;

1 ligne mise Ó jour.

EcoulÚ : 00 :00 :00.01
SQL> DELETE FROM Commande WHERE CodeCom = 2340729;

1 ligne supprimÚe.

EcoulÚ : 00 :00 :00.00
SQL> COMMIT;

Validation effectuÚe.

EcoulÚ : 00 :00 :00.00
SQL>
```

Fig. 7 : État initial des vues et modifications de la table Commande

```
SQL> SELECT CodeCom, Prix FROM VM1 WHERE CodeCom IN (99, 2340729, 2340931);

   CODECOM       PRIX
---------- ----------
        99      17099
   2340729        681

EcoulÚ : 00 :00 :00.00
SQL> SELECT CodeCom, Prix FROM VM2 WHERE CodeCom IN (99, 2340729, 2340931);

   CODECOM       PRIX
---------- ----------
        99      17099
   2340729        681

EcoulÚ : 00 :00 :00.00
SQL> SELECT mview_name, staleness FROM user_mviews;

MVIEW_NAME   STALENESS
------------ ---------------
VM1          NEEDS_COMPILE
VM2          NEEDS_COMPILE

EcoulÚ : 00 :00 :00.32
SQL> SELECT CodeCom, DMLTYPE$$, OLD_NEW$$ FROM MLOG$_COMMANDE;

   CODECOM D O
---------- - -
   2340931 I N
        99 U U
   2340729 D O

EcoulÚ : 00 :00 :00.00
SQL>
```

Fig. 8 : Vues non mises à jour et contenu du journal

Après le COMMIT, les deux vues n'ont pas changé : en mode ON DEMAND, rien n'est rafraîchi automatiquement, et Oracle les marque NEEDS_COMPILE. Le journal contient une ligne par opération : I (insertion), U (modification) et D (suppression).

```
SQL> SELECT CodeCom, Prix FROM VM1 WHERE CodeCom IN (99, 2340729, 2340931);

   CODECOM       PRIX
---------- ----------
        99      18099
   2340931       5000

EcoulÚ : 00 :00 :00.00
SQL> SELECT CodeCom, Prix FROM VM2 WHERE CodeCom IN (99, 2340729, 2340931);

   CODECOM       PRIX
---------- ----------
        99      18099
   2340931       5000

EcoulÚ : 00 :00 :00.00
SQL> SELECT mview_name, staleness, last_refresh_type FROM user_mviews;

MVIEW_NAME   STALENESS       LAST_REFRESH
------------ --------------- ------------
VM1          FRESH           COMPLETE
VM2          FRESH           FAST

EcoulÚ : 00 :00 :00.29
SQL> SELECT COUNT(*) FROM MLOG$_COMMANDE;

  COUNT(*)
----------
         0

EcoulÚ : 00 :00 :00.00
SQL>
```

Fig. 9 : Contenu des vues après rafraîchissement

Après rafraîchissement (`DBMS_MVIEW.REFRESH('VM1','C')` et `DBMS_MVIEW.REFRESH('VM2','F')`), les deux vues sont identiques et à jour, et le journal est vidé. Pour comparer les temps, nous avons refait un test identique (3 modifications), puis un test avec une modification massive de toutes les commandes de 2024.

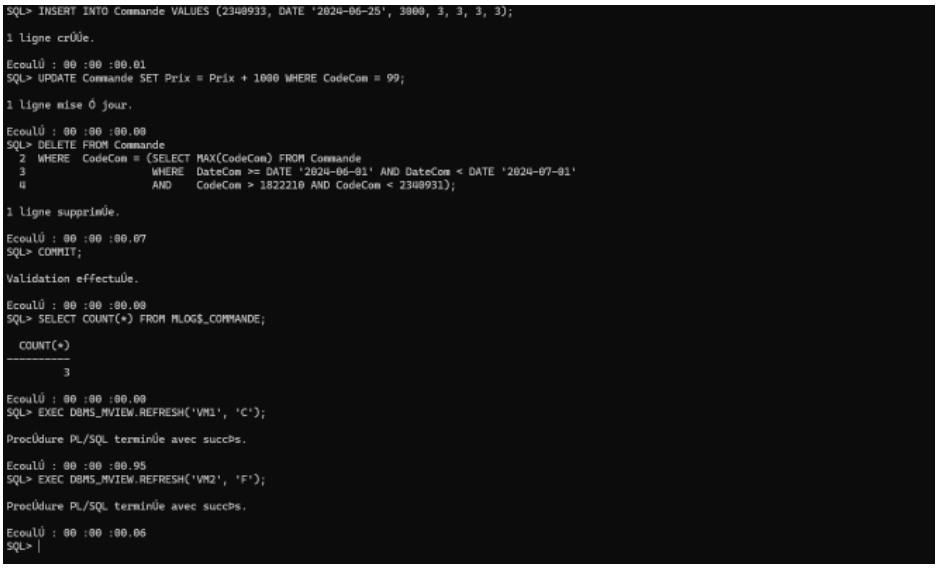


Fig. 10 : Temps de rafraîchissement après 3 modifications

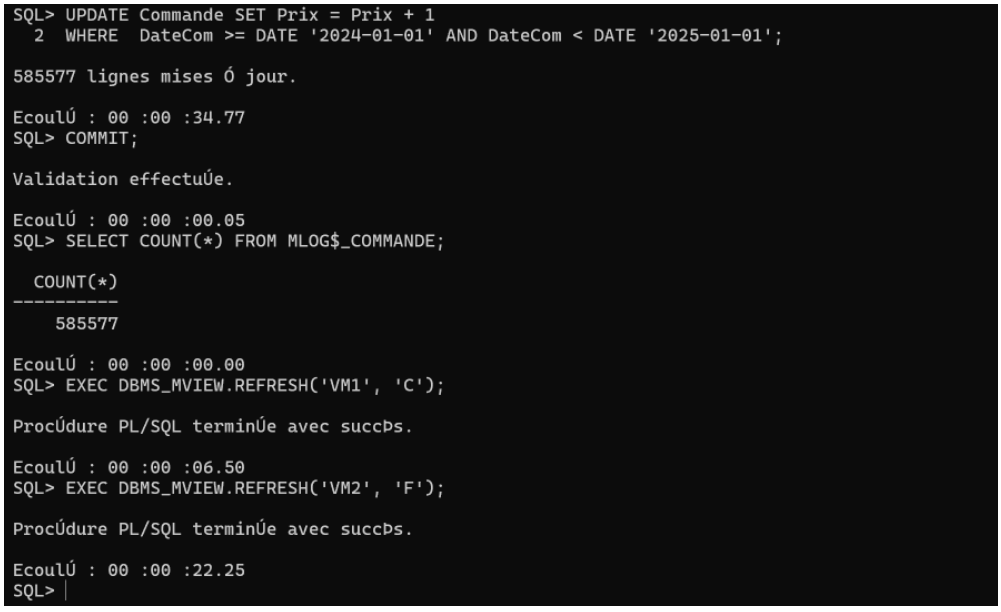


Fig. 11 : Temps de rafraîchissement après 585 577 modifications

	3 modifications	585 577 modifications
COMPLETE (VM1)	0,95 s	6,50 s
FAST (VM2)	0,06 s	22,25 s

Tab. 1 : Comparaison des temps de rafraîchissement

Avec peu de modifications, le FAST est environ 16 fois plus rapide : il n'applique que les 3 lignes du journal, alors que le COMPLETE relit toute la table Commande. Avec 585 577 modifications, c'est l'inverse : le coût du FAST augmente avec la taille du journal, qui contient en plus les modifications de tous les mois et pas seulement de juin, alors que le coût du COMPLETE reste à peu près constant. Le FAST est donc intéressant quand peu de lignes changent entre deux rafraîchissements. On note aussi que le journal ralentit les mises à jour de la table source (34,77 s pour l'UPDATE).

4 Journal de VM2 avant et après rafraîchissement

On insère la commande 2340932 (juin 2024), puis on consulte le journal avant et après le rafraîchissement de VM2.

```
SQL> COLUMN xid$$ FORMAT 999999999999999
SQL> SELECT CodeCom,
  2         TO_CHAR(SNAPTIME$$, 'DD/MM/YYYY') AS snaptime,
  3         DMLTYPE$$, OLD_NEW$$,
  4         RAWTOHEX(CHANGE_VECTOR$$) AS change_vector,
  5         XID$$
  6  FROM   MLOG$_COMMANDE;

   CODECOM SNAPTIME   D O
---------- ---------- - -
CHANGE_VECTOR
--------------------------------------------------------------------------------------------------------------------------------
           XID$$
---------------
   2340932 01/01/4000 I N
FE
 11259170867124 66


Ecoulé : 00 :00 :00.00
SQL> EXEC DBMS_MVIEW.REFRESH('VM2', 'F');

Procédure PL/SQL terminée avec succès.

Ecoulé : 00 :00 :00.23
SQL> SELECT * FROM MLOG$_COMMANDE;

   CODECOM SNAPTIME D O
---------- -------- - -
CHANGE_VECTOR$$
--------------------------------------------------------------------------------------------------------------------------------
           XID$$
---------------
   2340932 05/11/26 I N
FE
 11259170867124 66


Ecoulé : 00 :00 :00.00
SQL> SELECT CodeCom, Prix FROM VM2 WHERE CodeCom = 2340932;

   CODECOM       PRIX
---------- ----------
   2340932       7500

Ecoulé : 00 :00 :00.00
SQL>
```

Fig. 12 : Journal avant et après le rafraîchissement de VM2

Avant le rafraîchissement, le journal contient une ligne (I, N) avec une date SNAPTIME$$ au 01/01/4000, ce qui signifie qu'aucune vue ne l'a encore traitée, et la commande n'est pas dans VM2. Après le rafraîchissement, la commande est dans VM2, mais la ligne reste dans le journal, seule sa date SNAPTIME$$ a changé.

```
SQL> SELECT owner, master, mview_last_refresh_time, mview_id
  2  FROM   user_base_table_mviews;

OWNER    MASTER     MVIEW_LA   MVIEW_ID
-------- ---------- -------- ----------
MASTER   COMMANDE   05/11/26          2
MASTER   COMMANDE   05/11/26          1

Ecoulé : 00 :00 :00.12
SQL> SELECT master, mview_id,
  2         TO_CHAR(mview_last_refresh_time, 'DD/MM/YYYY HH24:MI:SS') AS dernier_refresh
  3  FROM   user_base_table_mviews;

MASTER       MVIEW_ID DERNIER_REFRESH
---------- ---------- -------------------
COMMANDE            2 05/11/2026 23:44:42
COMMANDE            1 05/11/2026 11:03:16

Ecoulé : 00 :00 :00.06
SQL> EXEC DBMS_MVIEW.REFRESH('VM1', 'C');

Procédure PL/SQL terminée avec succès.

Ecoulé : 00 :00 :02.43
SQL> SELECT COUNT(*) FROM MLOG$_COMMANDE;

  COUNT(*)
----------
         0

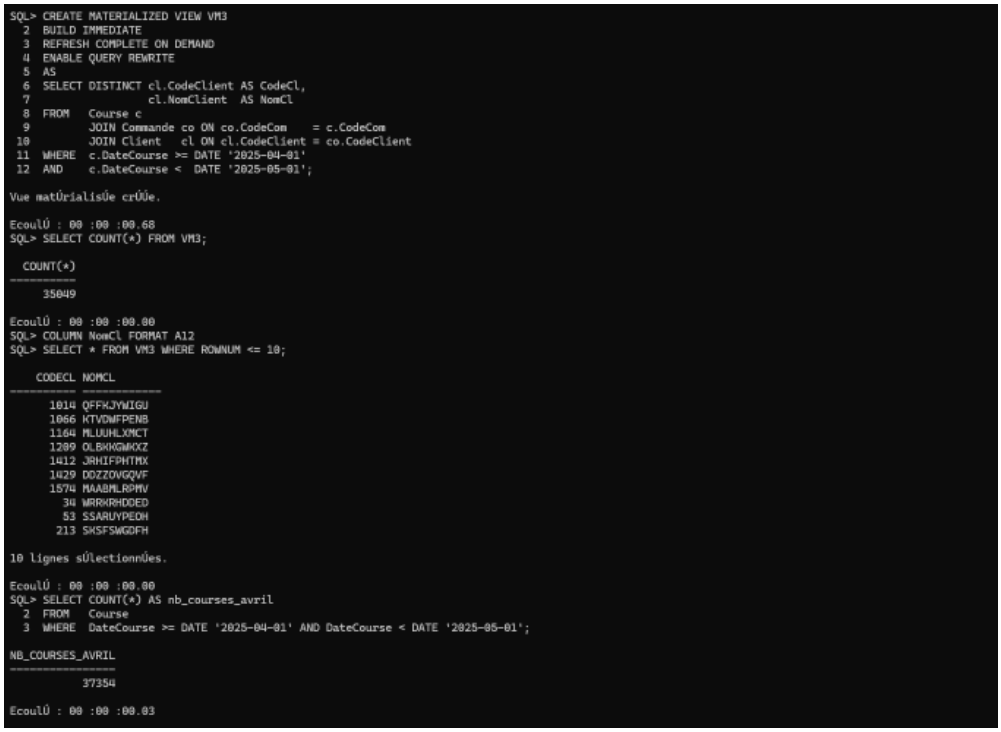
Ecoulé : 00 :00 :00.00
SQL>
```

Fig. 13 : Vues inscrites sur le journal et purge

L'explication est que deux vues sont inscrites sur ce journal : VM2, et VM1, qui s'y est inscrite lors de son rafraîchissement en question 3. VM1 n'ayant pas encore été rafraîchie depuis l'insertion, Oracle garde la ligne. Après le rafraîchissement de VM1, le journal est vide. Une ligne n'est donc purgée que lorsque toutes les vues inscrites l'ont prise en compte.

5 VM3 : clients ayant fait une course en avril 2025

La requête suit le chemin Course → Commande → Client avec des jointures (sans sous-requêtes imbriquées), filtre sur la date de la course et utilise DISTINCT pour ne garder chaque client qu'une fois.



```
SQL> CREATE MATERIALIZED VIEW VM3
  2  BUILD IMMEDIATE
  3  REFRESH COMPLETE ON DEMAND
  4  ENABLE QUERY REWRITE
  5  AS
  6  SELECT DISTINCT cl.CodeClient AS CodeCl,
  7                  cl.NomClient  AS NomCl
  8  FROM   Course c
  9         JOIN Commande co ON co.CodeCom    = c.CodeCom
 10         JOIN Client   cl ON cl.CodeClient = co.CodeClient
 11  WHERE  c.DateCourse >= DATE '2025-04-01'
 12  AND    c.DateCourse <  DATE '2025-05-01';

Vue matÚrialisÚe crÚÚe.

EcoulÚ : 00 :00 :00.68
SQL> SELECT COUNT(*) FROM VM3;

  COUNT(*)
----------
     35049

EcoulÚ : 00 :00 :00.00
SQL> COLUMN NomCl FORMAT A12
SQL> SELECT * FROM VM3 WHERE ROWNUM <= 10;

    CODECL NOMCL
---------- ------------
      1014 QFFKJYWIGU
      1066 KTVDWFPENB
      1164 MLUUHLXMCT
      1209 OLBKHGWKXZ
      1412 JRHIFPHTMX
      1429 DDZZOVGQVF
      1574 MAABMLRPMV
        34 WRRKRHDDED
        53 SSARUYPEOH
       213 SHSFSWGDFH

10 lignes sÚlectionnÚes.

EcoulÚ : 00 :00 :00.00
SQL> SELECT COUNT(*) AS nb_courses_avril
  2  FROM   Course
  3  WHERE  DateCourse >= DATE '2025-04-01' AND DateCourse < DATE '2025-05-01';

NB_COURSES_AVRIL
----------------
           37354

EcoulÚ : 00 :00 :00.03
```

Fig. 14 : Création et contenu de VM3

VM3 contient 35 049 clients pour 37 354 courses : certains clients ont fait plusieurs courses dans le mois.

6 VM4 : nombre de courses par mois

```
SQL> CREATE MATERIALIZED VIEW VM4
  2  BUILD IMMEDIATE
  3  REFRESH COMPLETE ON DEMAND
  4  ENABLE QUERY REWRITE
  5  AS
  6  SELECT EXTRACT(MONTH FROM DateCourse) AS Mois,
  7         EXTRACT(YEAR  FROM DateCourse) AS Annee,
  8         COUNT(*)                       AS NBCourses
  9  FROM   Course
 10  GROUP BY EXTRACT(YEAR FROM DateCourse), EXTRACT(MONTH FROM DateCourse);

Vue matÚrialisÚe crÚÚe.

EcoulÚ : 00 :00 :00.42
SQL> SELECT COUNT(*) AS nb_lignes FROM VM4;

 NB_LIGNES
----------
        49

EcoulÚ : 00 :00 :00.00
SQL> SELECT * FROM VM4 ORDER BY Annee, Mois;

      MOIS      ANNEE  NBCOURSES
---------- ---------- ----------
         1       2022      37693
         2       2022      34574
         3       2022      38471
         4       2022      37175
         5       2022      38633
         6       2022      37133
         7       2022      38940
         8       2022      38496
         9       2022      37747
        10       2022      38943
        11       2022      37435
        12       2022      38615
         1       2023      38559
         2       2023      35242
         3       2023      38746
         4       2023      37655
         5       2023      38416
         6       2023      37327
         7       2023      38281
         8       2023      38742
         9       2023      37746
        10       2023      39118
        11       2023      37577
        12       2023      39266
         1       2024      38266
```

Fig. 15 : Création de VM4

```
         1       2024      38266
         2       2024      35949
         3       2024      38418
         4       2024      37127
         5       2024      38993
         6       2024      37466
         7       2024      38741
         8       2024      38514
         9       2024      37523
        10       2024      38778
        11       2024      37542
        12       2024      38536
         1       2025      38794
         2       2025      34711
         3       2025      38729
         4       2025      37354
         5       2025      38485
         6       2025      37316
         7       2025      38687
         8       2025      38984
         9       2025      37560
        10       2025      38467
        11       2025      37399
        12       2025      38261
         1       2026       1240

49 lignes sÚlectionnÚes.

EcoulÚ : 00 :00 :00.01
SQL> SELECT SUM(NBCourses) AS total FROM VM4;

     TOTAL
----------
   1822210
```

Fig. 16 : Suite du résultat et total

VM4 contient 49 lignes et un total de 1 822 210 courses. Février est toujours le mois le plus faible (moins de jours), avec un peu plus de courses en 2024, année bissextile. La ligne de janvier 2026 (1 240 courses) vient des commandes de fin décembre 2025, puisqu'une course peut avoir lieu jusqu'à deux jours après sa commande.

7 VM3 et VM4 en mode FAST

Le problème

```
SQL> CREATE MATERIALIZED VIEW VM3_FAST
  2  BUILD IMMEDIATE
  3  REFRESH FAST ON DEMAND
  4  AS
  5  SELECT DISTINCT cl.CodeClient AS CodeCl,
  6                  cl.NomClient  AS NomCl
  7  FROM   Course c
  8         JOIN Commande co ON co.CodeCom    = c.CodeCom
  9         JOIN Client   cl ON cl.CodeClient = co.CodeClient
 10  WHERE  c.DateCourse >= DATE '2025-04-01'
 11  AND    c.DateCourse <  DATE '2025-05-01';
AND    c.DateCourse <  DATE '2025-05-01'
                       *
ERREUR Ó la ligne 11 :
ORA-12015: impossible de crÚer une vue matÚrialisÚe de rÚgÚnÚration de type Fast Ó partir d'une
requÛte complexe


EcoulÚ : 00 :00 :00.01
SQL> CREATE MATERIALIZED VIEW VM4_FAST
  2  BUILD IMMEDIATE
  3  REFRESH FAST ON DEMAND
  4  AS
  5  SELECT EXTRACT(MONTH FROM DateCourse) AS Mois,
  6         EXTRACT(YEAR  FROM DateCourse) AS Annee,
  7         COUNT(*)                       AS NBCourses
  8  FROM   Course
  9  GROUP BY EXTRACT(YEAR FROM DateCourse), EXTRACT(MONTH FROM DateCourse);
FROM   Course
       *
ERREUR Ó la ligne 8 :
ORA-23413: la table "MASTER"."COURSE" n'a pas de journal des vues matÛrialisÚes


EcoulÚ : 00 :00 :00.00
SQL>
```

Fig. 17 : Tentative de création de VM3 et VM4 en FAST

```
SQL> SELECT mvname, capability_name, possible, msgtxt
  2  FROM   mv_capabilities_table
  3  WHERE  capability_name LIKE 'REFRESH_FAST%'
  4  ORDER BY mvname, seq;

MVNAME CAPABILITY_NAME              POS MSGTXT
------ ------------------------------ --- --------------------------------------------------------------
VM3    REFRESH_FAST                 N
VM3    REFRESH_FAST_AFTER_INSERT    N   les sous-requÛtes ou les vues incorporÚes dans la liste FROM
                                        ne sont pas prises en charge pour cette vue matÛrialisÚe

VM3    REFRESH_FAST_AFTER_INSERT    N   les sous-requÛtes ou les vues incorporÚes dans la liste FROM
                                        ne sont pas prises en charge pour cette vue matÛrialisÚe

VM3    REFRESH_FAST_AFTER_INSERT    N   vue ou sous-requÛte depuis liste
VM3    REFRESH_FAST_AFTER_INSERT    N   clause DISTINCT dans liste SELECT dans vue matÛrialisÚe
VM3    REFRESH_FAST_AFTER_ONETAB_DM N   vÚrifiez le motif de la dÚsactivation de REFRESH_FAST_AFTER_
       L                                INSERT

VM3    REFRESH_FAST_AFTER_ANY_DML   N   vÚrifiez le motif de la dÚsactivation de REFRESH_FAST_AFTER_
                                        ONETAB_DML

VM3    REFRESH_FAST_PCT             N   PCT FAST REFRESH impossible si la requÛte contient une vue i
                                        ncorporÚe

VM4    REFRESH_FAST                 N
VM4    REFRESH_FAST_AFTER_INSERT    N   la table dÚtails ne comprend pas de journal de vue matÛriali
                                        sÚe

VM4    REFRESH_FAST_AFTER_ONETAB_DM N   vÚrifiez le motif de la dÚsactivation de REFRESH_FAST_AFTER_
       L                                INSERT

VM4    REFRESH_FAST_AFTER_ANY_DML   N   vÚrifiez le motif de la dÚsactivation de REFRESH_FAST_AFTER_
                                        ONETAB_DML

VM4    REFRESH_FAST_PCT             N   PCT impossible dans les tables dÚtails de la vue matÛrialisÚ
                                        e


13 lignes sÚlectionnÚes.

EcoulÚ : 00 :00 :00.01
SQL>
```

Fig. 18 : Diagnostic avec DBMS_MVIEW.EXPLAIN_MVIEW

Les deux créations échouent pour des raisons différentes :

- **VM3** (ORA-12015) : le DISTINCT est interdit en FAST, car lors d'une suppression Oracle ne peut pas savoir si le client doit sortir de la vue. De plus, EXPLAIN_MVIEW montre que la syntaxe `JOIN ... ON` est transformée en vues incorporées, que le FAST ne supporte pas.
- **VM4** (ORA-23413) : la requête est acceptée, mais la table Course n'a pas de journal.

La solution

Nous avons créé des journaux plus complets (ROWID, SEQUENCE, colonnes utilisées et INCLUDING NEW VALUES) sur Course et Client, et complété celui de Commande avec `ALTER MATERIALIZED VIEW LOG` pour ne pas casser VM2. VM4 est recréée telle quelle. Pour VM3, les jointures sont écrites dans le WHERE et le DISTINCT est remplacé par un GROUP BY avec un COUNT(*), qui donne toujours un client par ligne mais permet à Oracle de savoir quand retirer un client.

```
SQL> CREATE MATERIALIZED VIEW LOG ON Course
  2    WITH ROWID, SEQUENCE (DateCourse, CodeCom)
  3    INCLUDING NEW VALUES;

Journal de vue matÚrialisÚe crÚÚ.

EcoulÚ : 00 :00 :00.02
SQL> CREATE MATERIALIZED VIEW LOG ON Client
  2    WITH ROWID, SEQUENCE (CodeClient, NomClient)
  3    INCLUDING NEW VALUES;

Journal de vue matÚrialisÚe crÚÚ.

EcoulÚ : 00 :00 :00.01
SQL> ALTER MATERIALIZED VIEW LOG ON Commande
  2    ADD ROWID, SEQUENCE (CodeClient)
  3    INCLUDING NEW VALUES;

Journal de vue matÚrialisÚe modifiÚ.

EcoulÚ : 00 :00 :00.04
SQL> CREATE MATERIALIZED VIEW VM4_FAST
  2  BUILD IMMEDIATE
  3  REFRESH FAST ON DEMAND
  4  ENABLE QUERY REWRITE
  5  AS
  6  SELECT EXTRACT(MONTH FROM DateCourse) AS Mois,
  7         EXTRACT(YEAR  FROM DateCourse) AS Annee,
  8         COUNT(*)                       AS NBCourses
  9  FROM   Course
 10  GROUP BY EXTRACT(YEAR FROM DateCourse), EXTRACT(MONTH FROM DateCourse);

Vue matÚrialisÚe crÚÚe.

EcoulÚ : 00 :00 :00.52
SQL> CREATE MATERIALIZED VIEW VM3_FAST
  2  BUILD IMMEDIATE
  3  REFRESH FAST ON DEMAND
  4  ENABLE QUERY REWRITE
  5  AS
  6  SELECT cl.CodeClient AS CodeCl,
  7         cl.NomClient  AS NomCl,
  8         COUNT(*)      AS NbCourses
  9  FROM   Course c, Commande co, Client cl
 10  WHERE  co.CodeCom    = c.CodeCom
 11  AND    cl.CodeClient = co.CodeClient
 12  AND    c.DateCourse >= DATE '2025-04-01'
 13  AND    c.DateCourse <  DATE '2025-05-01'
 14  GROUP BY cl.CodeClient, cl.NomClient;

Vue matÚrialisÚe crÚÚe.
```

FIG. 19 : Journaux et création de VM4_FAST et VM3_FAST

```
SQL> SELECT COUNT(*) FROM VM3_FAST;

  COUNT(*)
----------
     35049

EcoulÚ : 00 :00 :00.00
SQL> SELECT COUNT(*) FROM VM4_FAST;

  COUNT(*)
----------
        49

EcoulÚ : 00 :00 :00.00
SQL> SELECT mview_name, refresh_method, fast_refreshable FROM user_mviews;

MVIEW_NAME   REFRESH_ME FAST_REFRESHABLE
------------ ---------- ----------------
VM1          COMPLETE   DIRLOAD_DML
VM2          FAST       DIRLOAD_DML
VM3          COMPLETE   NO
VM4          COMPLETE   DIRLOAD_DML
VM4_FAST     FAST       DIRLOAD_DML
VM3_FAST     FAST       DIRLOAD_DML

6 lignes sÚlectionnÚes.

EcoulÚ : 00 :00 :00.11
SQL>
```

FIG. 20 : Contenu et mode de rafraîchissement des vues

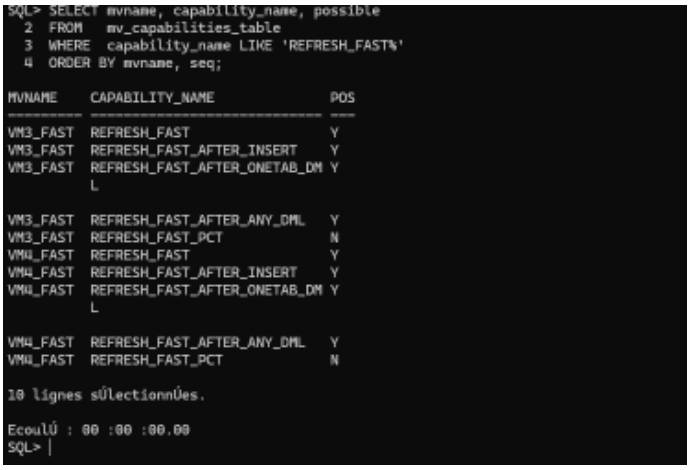


FIG. 21 : Capacités FAST des nouvelles vues

Les nouvelles vues ont le même contenu que VM3 et VM4, et EXPLAIN_MVIEW confirme qu'elles sont rafraîchissables en FAST après tout type de modification. Le PCT reste impossible car nos tables ne sont pas partitionnées.

Vérification

Nous avons ajouté une course en avril 2025, puis rafraîchi les quatre vues.

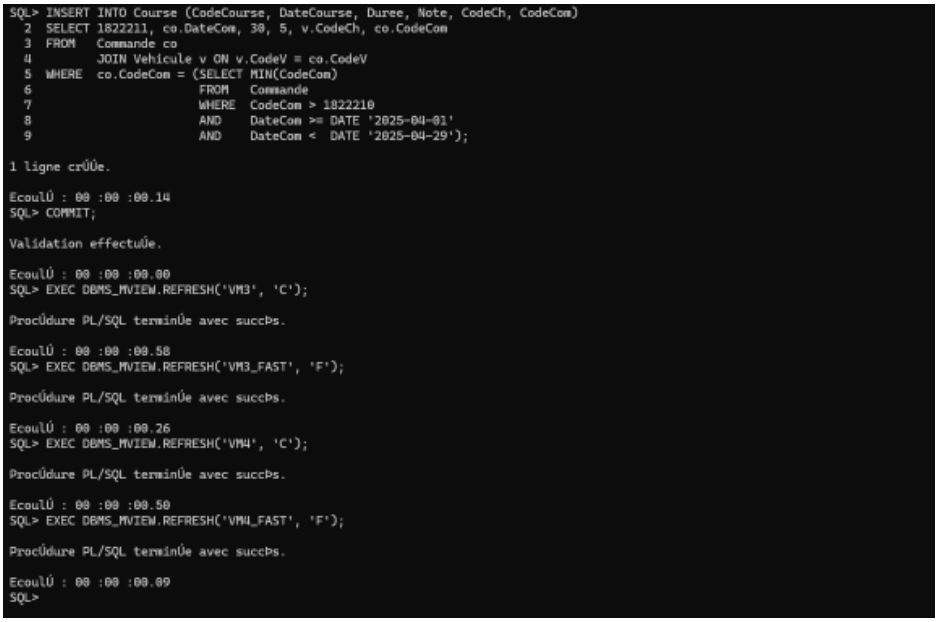


FIG. 22 : Ajout d'une course et rafraîchissements

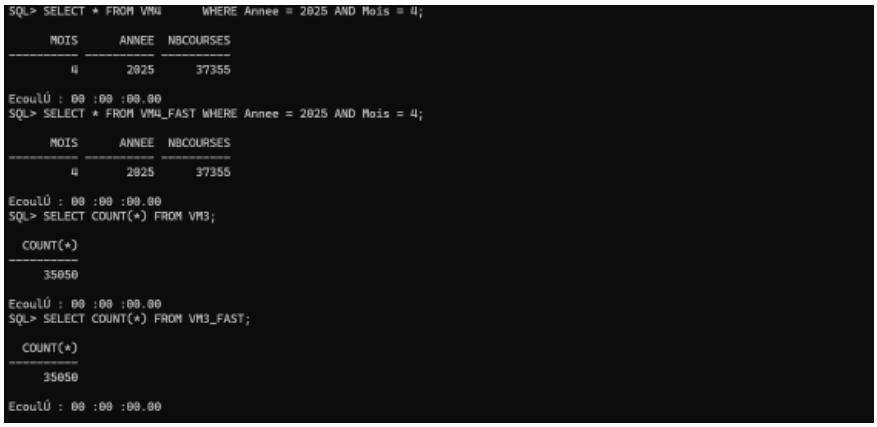


Fig. 23 : Résultats après rafraîchissement

	COMPLETE	FAST
VM3	0,58 s	0,26 s
VM4	0,50 s	0,09 s

Tab. 2 : Temps de rafraîchissement de VM3 et VM4

Les versions COMPLETE et FAST donnent le même résultat (37 355 courses en avril 2025, 35 050 clients), et le FAST est plus rapide. Le gain est plus faible que pour VM2 car Oracle doit maintenir des compteurs et, pour VM3, refaire les jointures pour chaque ligne modifiée.

Conclusion

Les vues matérialisées évitent de recalculer des requêtes coûteuses, au prix d'un rafraîchissement. Le mode COMPLETE est simple et son coût ne dépend pas du nombre de modifications ; le mode FAST est bien plus rapide quand peu de lignes changent, mais il demande des journaux sur les tables sources et impose des contraintes sur la requête (pas de DISTINCT, jointures dans le WHERE, COUNT(*) pour les agrégats). Ce sont exactement les mécanismes utilisés dans un entrepôt de données pour maintenir des tables d'agrégats comme VM4.